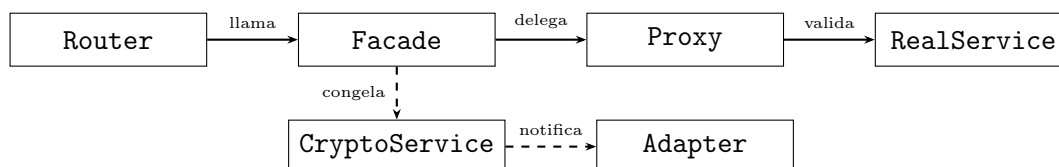


Patrones Estructurales Implementados

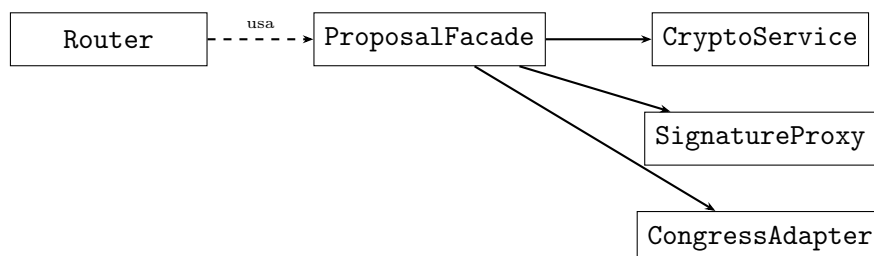
Plataforma Voz del Ciudadano

Se implementaron cinco patrones estructurales. El siguiente diagrama muestra el flujo completo cuando un ciudadano firma una propuesta:



1. Facade — patterns/facade.py

Cuando el router necesita crear o firmar una propuesta, hay varios subsistemas involucrados: la base de datos, el servicio de hash, el adaptador del Congreso y la auditoría. En vez de que el router sepa de todos ellos, `ProposalFacade` los agrupa y expone solo tres métodos. El router llama uno de esos métodos y el Facade se encarga del resto internamente.



```

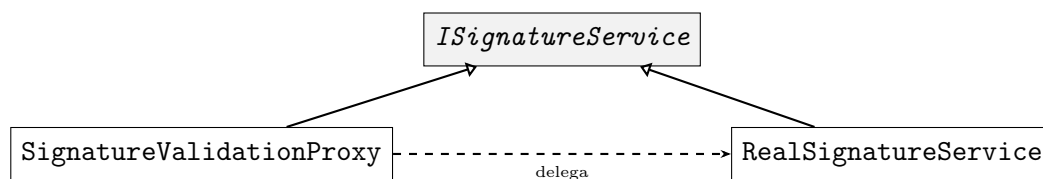
# routers/proposals.py -> instancia ProposalFacade con la sesion de BD
facade = ProposalFacade(db) # recibe Session de SQLAlchemy
sig = facade.firmar_propuesta(proposal_id, current_user.id)

# patterns/facade.py -> constructor: inicializa todos los subsistemas
def __init__(self, db: Session):
    self._db = db # Session (models: Proposal, Signature)
    self._crypto = CryptoService() # genera hash SHA-256
    self._congress = CongressNotificationAdapter() # Adapter
    self._congress_audited = AuditServiceDecorator( # Decorator sobre Adapter
        self._congress, "CongressAdapter"
    )
    self._signature_proxy = SignatureValidationProxy( # Proxy sobre RealService
        RealSignatureService()
    )
  
```

```
# patterns/facade.py -> firmar_propuesta: coordina Proxy, modelos y
CryptoService
def firmar_propuesta(self, proposal_id, user_id):
    sig = self._signature_proxy.firmar(proposal_id, user_id, self._db)
    proposal = self._db.query(Proposal).filter_by(id=proposal_id).first()
    if proposal.firmas_count >= FIRMA_LIMITE:      # FIRMA_LIMITE viene de
models.py
        self._congelar_y_enviar(proposal)          # activa CryptoService +
Adapter
    return sig
```

2. Proxy — patterns/proxy.py

Antes de guardar una firma hay que verificar que la propuesta esté activa, que no haya vencido el plazo y que el usuario no haya firmado antes. Esa validación vive en `SignatureValidationProxy`, que tiene la misma interfaz que el servicio real. Si algo no pasa, corta ahí y el servicio real nunca llega a ejecutarse.



```
# patterns/facade.py -> el Facade inyecta RealSignatureService en el
Proxy
self._signature_proxy = SignatureValidationProxy(RealSignatureService())

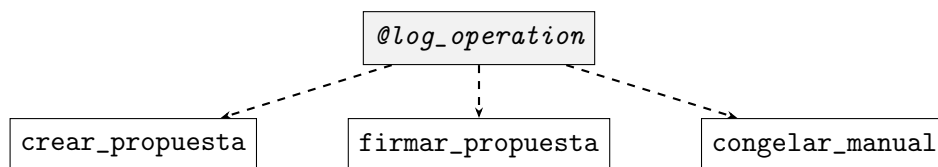
# patterns/proxy.py -> consulta el modelo Proposal y Signature de
models.py
def firmar(self, proposal_id, user_id, db):
    proposal = db.query(Proposal).filter_by(id=proposal_id).first()
    if proposal.estado != "activa":                # campo del modelo
Proposal
        raise HTTPException(400, "...")
    if datetime.now() > proposal.deadline:         # campo deadline de
Proposal
        raise HTTPException(400, "...")
    existing = db.query(Signature).filter_by(      # consulta modelo
Signature
        proposal_id=proposal_id, user_id=user_id
    ).first()
    if existing:
        raise HTTPException(409, "Ya has firmado esta propuesta")
    return self._real.firmar(proposal_id, user_id, db) # delega a
RealSignatureService

# patterns/proxy.py -> RealSignatureService crea el hash y actualiza
Proposal
```

```
def firmar(self, proposal_id, user_id, db):    # RealSignatureService
    hash_firma = sha256(f"{user.dni}:{proposal_id}:{ts}").hexdigest()
    db.add(Signature(..., hash_firma=hash_firma))    # crea registro
    Signature
    proposal.firmas_count += 1                    # actualiza contador
    en Proposal
    db.commit()
```

3. Decorator — patterns/decorator.py

Se necesitaba registrar en un log cada vez que se crea, firma o congela una propuesta, sin tener que repetir ese código en cada método. `@log_operation` se pone encima del método y graba el timestamp y si terminó bien o con error. `AuditServiceDecorator` hace lo mismo pero envolviendo el objeto del Adapter del Congreso.



```
# patterns/facade.py -> @log_operation decora los 3 metodos publicos
# del Facade
@log_operation("CREAR_PROPOSTA")    # antes y despues registra en
    _audit_log
def crear_propuesta(self, ...): ...

@log_operation("FIRMAR_PROPOSTA")    # captura errores del Proxy tambien
def firmar_propuesta(self, ...): ...

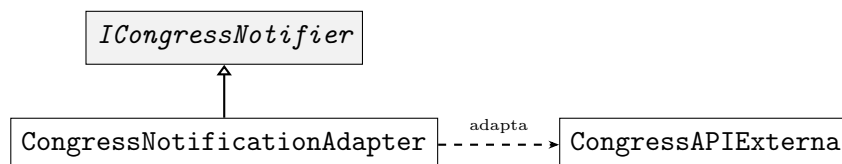
@log_operation("CONGELAR_MANUAL")
def congelar_manual(self, ...): ...

# patterns/facade.py -> AuditServiceDecorator envuelve
# CongressNotificationAdapter
self._congress_audited = AuditServiceDecorator(
    CongressNotificationAdapter(), "CongressAdapter"
)
# cada llamada a enviar_iniciativa queda registrada en _audit_log (
# decorator.py)
resultado = self._congress_audited.enviar_iniciativa(datos)

# routers/proposals.py -> el audit log es consultable via endpoint
@router.get("/admin/audit-log")
def audit_log():
    return get_audit_log()    # retorna _audit_log de patterns/decorator.
    py
```

4. Adapter — patterns/adapter.py

La API del Congreso recibe los datos con nombres de parámetros distintos a los que usa el sistema internamente. `CongressNotificationAdapter` hace esa traducción: toma el diccionario interno y lo convierte al formato que espera `CongressAPIExterna`, incluyendo las comisiones parlamentarias que se asignan según el título de la propuesta.



```

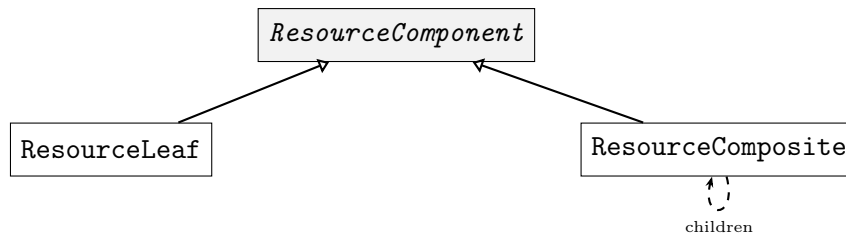
# patterns/facade.py -> _congelar_y_enviar llama al Adapter con
# formato interno
hash_value = self._crypto.freeze(proposal, signatures) # CryptoService
# -> SHA-256
resultado = self._congress_audited.enviar_iniciativa({ # dict interno
# del dominio
    "titulo":      proposal.titulo,      # campo del modelo Proposal
    "contenido":   proposal.contenido,
    "firmas_count": proposal.firmas_count,
    "hash":        hash_value,
})
# si exito: actualiza proposal.estado y proposal.tracking_congreso en BD

# patterns/adapter.py -> CongressNotificationAdapter traduce a
# CongressAPIExterna
def enviar_iniciativa(self, datos: dict) -> dict:
    return self._api.submit_citizen_initiative( # metodo de
        CongressAPIExterna
        initiative_title = datos["titulo"],      # renombrado
        initiative_body   = datos["contenido"],  # renombrado
        total_signatures  = datos["firmas_count"], # renombrado
        integrity_hash     = datos["hash"],
        submission_date    = datetime.now().isoformat(), # campo nuevo
        committees         = self._determinar_comisiones(datos["titulo"])
        # calculado
    )

```

5. Composite — patterns/composite.py

Los recursos se guardan en la BD como registros planos con `parent_id`, pero en el frontend se muestran anidados: un comentario puede tener respuestas, una modificación puede tener comentarios, y así. El `ResourceTreeBuilder` toma esa lista plana y la convierte en un árbol. Tanto `ResourceLeaf` como `ResourceComposite` responden al mismo método `to_dict()`, entonces el builder no necesita distinguir entre ellos para serializar.



```

# routers/proposals.py -> carga registros planos del modelo Resource (
models.py)
resources = db.query(Resource).filter_by(proposal_id=proposal_id).all()
builder = ResourceTreeBuilder()
return builder.build(resources)      # retorna lista de dicts anidados al
frontend

# patterns/composite.py -> ResourceTreeBuilder lee los campos de
Resource
def build(self, db_resources: list) -> list[dict]:
    for r in db_resources:
        autor = r.user.nombre          # relacion Resource -> User (
models.py)
        has_children = bool(r.children) # relacion self-referencial de
Resource
        if has_children:
            node = ResourceComposite(r.id, r.tipo, r.contenido, autor,
...)
        else:
            node = ResourceLeaf(r.id, r.tipo, r.contenido, autor, ...)
        nodes[r.id] = node

    for r in db_resources:
        if r.parent_id is None:         # campo parent_id de Resource (
models.py)
            roots.append(nodes[r.id])
        else:
            parent.add(nodes[r.id])     # add() solo existe en
ResourceComposite

# ResourceComposite serializa recursivamente todos sus hijos
def to_dict(self):
    return { ..., "children": [c.to_dict() for c in self._children] }

```